



Department of Computer Science and Engineering
Faculty of Automatic Control and Computer Science
National University of Science and Technology
POLITEHNICA Bucharest



Bachelor Thesis

Titlu lucrare

Supervisor
Assist. Prof. PhD Eng. Ștefan-Dan
Ciocîrlan

Student
El Studento

2026

Contents

Abstract	1
1. Introduction	2
2. Background	4
Alternative approaches and related work	5
Goal and contributions	6
TLS 1.3 handshake	7
Security models and information-theoretic primitives	8
Classical key exchange in TLS	10
The quantum threat	10
Post-quantum cryptography	11
Quantum foundations	12
The BB84 protocol	12
The ETSI QKD 014 key delivery standard	14
Threat model	14
3. Architecture	15
Solution overview	15
Functional decomposition	16
Secure application entity (SAE)	16
Key management entity (KME)	17
QKD endpoint	17
Benchmark module	17
Visualisation dashboard	18
Interfaces and communication	18
Non-functional considerations	19
Mapping to implementation and evaluation	19
4. Implementation	19
Implementation overview	19
How the stack is assembled	20
Technology choices	20
Core components	22
Key management entity	22
QKD layer	24
QTLS endpoint	25
Environment and setup	26
5. Evaluation	27
Evaluation strategy	27
Benchmarks and metrics	27
Experimental setup	28
Results	28
Handshake latency	28
Throughput	29

Pool drain	29
Key consumption	30
Key rate versus QBER	31
Key exchange comparison	31
Data analysis	32
Evaluation conclusion	33
6. Discussion	33
Summary of key findings	33
Interpretation of results	33
Comparison with existing work	34
Implications for the domain	34
Limitations	35
Future work	35
7. Conclusions	36
References	37
Appendices	40

Abstract

ABSTRACT WRITING GUIDE (sentence by sentence)

1. Domain -one sentence: broad field of the thesis. e.g., “This thesis lies in the field of computer architecture and hardware accelerators.” e.g., “This thesis lies in the field of scientific computing and high-performance numerical methods.”
2. Context -one sentence: specific setting of your work. e.g., “We focus on number-representation systems implemented in hardware.” e.g., “We focus on software libraries for number representations used in large-scale scientific-computing workflows.”
3. Problem -one sentence: the gap or deficiency you address. e.g., “Currently there is no unified hardware library that implements multiple number representations with a common benchmark suite.” e.g., “There is no existing software library that provides diverse number representations together with automated testing and benchmarking.”
4. Importance (optional but recommended) -one sentence: why it matters. e.g., “Choosing an appropriate number format can reduce silicon area, lower energy consumption, and improve computational accuracy.” e.g., “Using the right number format can increase numerical precision, reduce runtime, and lower energy usage.”
5. Solution -one sentence: what you built or contributed. e.g., “We present a parameterizable Chisel library that implements fixed-point, floating-point, posit, and unum formats, together with a shared test-and-benchmark framework.” e.g., “We provide a Scala-based software library integrated into a Jenkins CI pipeline that offers the same set of number representations and automatically runs correctness and performance tests.”
6. Alternative solutions (optional) -one sentence: what exists and what it lacks. e.g., “Existing tools such as FloPoCo focus mainly on floating-point and lack support for newer formats like posits.” e.g., “General-purpose libraries (e.g., NumPy, Julia’s standard library) provide many formats but do not enforce automated testing or CI integration for reproducible evaluation.”
7. Key result -one sentence: most important outcome of your evaluation. e.g., “In our FPGA experiments, the posit-based multiplier achieved 30% lower error than IEEE-754 at comparable resource usage.” e.g., “Benchmarking shows that, for a climate-model kernel, the software posit implementation reduces mean absolute error by 25% versus double-precision while keeping runtime within 5%.”
8. Conclusion -one sentence: what your work enables. e.g., “The resulting benchmark suite lets hardware designers select number representations based on concrete trade-offs between accuracy, area, and power.” e.g., “The library and Jenkins workflow give scientific-computing teams an easy way to experiment with alternative number formats and quantify their impact on correctness and cost.”
9. Impact on the field (optional) -one sentence: broader influence. e.g., “This can lower production costs for numerical accelerators and encourage wider adoption of efficient,

application-specific formats.” e.g., “It promotes more reproducible scientific software by making number-representation choice a first-class, testable concern in CI pipelines.”

Replace the block below with your 7-9 sentence abstract, following the order above.

1. Introduction

Modern society depends on digital communication for nearly every critical activity, from banking and healthcare to government services and personal correspondence containing private information. Securing this communication became an important concern of computer science, and the dominant protocol that addresses it is Transport Layer Security (TLS), in its latest version 1.3 [1]. The adoption of encrypted transport has become the norm: as of 2026, around 95% of the web traffic observed by major browsers is protected by HTTPS, which has only grown in recent years [2]. The security guarantees that this infrastructure provides (confidentiality, integrity, and authentication) rely, however, on a specific class of mathematical assumptions: that certain problems, such as factoring large integers or computing discrete logarithms over elliptic curves, are computationally infeasible for any realistic adversary [3].

The most security critical step of a TLS 1.3 session is the initial key exchange, since every subsequent record on the connection is based on keys derived from it. TLS 1.3 performs this exchange with ephemeral Diffie-Hellman, in both its finite-field and elliptic-curve variants, and modern implementations use the elliptic-curve form (ECDHE) over the standardised curves X25519 and P-256 [1], [4]. The security of ECDHE reduces to the Elliptic-Curve Discrete Logarithm Problem (ECDLP), introduced by Koblitz in the mid-1980s as the basis for elliptic-curve cryptosystems [5]. At the parameter sizes recommended by NIST, no known classical algorithm solves ECDLP in less than exponential time, which is why TLS 1.3 is considered secure today. However this is a strong assumption about the computational model available to the attacker.

That assumption breaks under quantum computation. Shor’s algorithm solves both integer factoring and the discrete logarithm problem, including its elliptic-curve variant, in polynomial time on a sufficiently large quantum computer [6]. Today it is not the case, but every public-key primitive currently deployed in TLS 1.3 (RSA, finite-field DH, and ECDHE) loses its key-exchange security when such computers become available. Furthermore, the threat is not purely a future one. An attacker who records encrypted traffic today and stores it can decrypt it later. This “harvest now, decrypt later” model makes the quantum threat a problem of the present for any data that must remain confidential for decades, such as state secrets, medical records, and industrial intellectual property.

It is important to understand that the urgency of this threat is not the same across applications. For short-lived data, such as a one-off web session or a disappearing chat, a moderately delayed quantum break would arrive long after the data has lost any value. For data with a long secrecy horizon, any quantum computer that arrives before the secrecy requirement expires compromises the data. Mosca formalised this trade-off as a simple inequality: if data must remain secret for X years, and deploying a cryptographic stack for that data takes Y

years, then a quantum computer arriving in fewer than $X + Y$ years is already a concern today [7]. For documents whose required secrecy lifetime is measured in decades, such as classified government communications, health and legal records, or industrial trade secrets, this inequality is plausibly already violated.

The categories of data for which Mosca’s inequality is already violated are not hypothetical. Electronic health records are required by national legislation to remain confidential for the lifetime of the patient and beyond, and the European Union’s General Data Protection Regulation imposes similar long-term confidentiality obligations on any personal data that has been collected. Classified government communications follow declassification schedules measured in decades, with NATO COSMIC TOP SECRET and equivalent national markings often requiring secrecy for fifty years or more. Industrial trade secrets and pharmaceutical research data lose commercial value only when patents expire, which is typically twenty years after filing. For all of these categories, an adversary who records ciphertext today and decrypts it ten or twenty years from now would still cause harm.

The used solution to the quantum threat in TLS today is to enforce the protocol with post-quantum cryptography (PQC), a family of public-key primitives that replaces ECDLP with newer mathematical problems such as structured lattices [3]. The approach is a hybrid key exchange: the client and server run ECDHE over X25519 and ML-KEM-768 [8] in parallel, and combine the two shared secrets. This solution is used by Cloudflare and Google. They have enabled X25519MLKEM768 by default on a significant fraction of TLS 1.3 traffic since 2024. Two limitations matter for this thesis. First, PQC remains a computational construction: the lattice problems it relies on have been studied for far less time than factoring or discrete logarithms, and a future algorithmic advance, classical or quantum, could weaken any of them, so for data with multi-decade secrecy horizons the residual risk does not disappear. Second, only the key-exchange step is touched: the record layer still encrypts data with AES-GCM, so even with PQC key agreement the data path is not information-theoretically secure.

On the QKD side of the market, vendors such as ID Quantique, Toshiba, and QuantumX-change ship QKD-VPN and QKD-Ethernet products that use quantum-distributed keys to seed AES-256 for the data path. This fixes the key-exchange step but leaves the bulk encryption computational, so the stack is again not information-theoretically secure end to end.

This thesis presents a QTLS software simulation, a TLS 1.3 shaped protocol whose data path is information-theoretically secure end to end. The TLS 1.3 architecture is preserved (the handshake, key schedule, record layer, and close), with replacements that close this gap: ECDHE becomes a Quantum Key Distribution (QKD) key fetched by `key_ID` over the ETSI GS QKD 014 REST API [9], AES-GCM becomes a One-Time Pad [10], and HMAC becomes a Wegman-Carter MAC over $GF(2^{128})$ [11]. The QKD layer is a Qiskit-based BB84 simulation, followed by Cascade reconciliation with back-propagation and privacy amplification [12] the quantum channel is modelled as an abstract link with a configurable error rate, and the Key Management Entity (KME) runs as a local service that exposes the standard ETSI 014 HTTP API. The result is a reproducible reference in which every part of the implementation is information-theoretically secure.

QTLS is evaluated along two axes. The first is correctness: a deterministic test suite covering BB84 sifting, Cascade convergence, the ETSI 014 KME contract, the OTP and Wegman-Carter primitives, the full QTLS handshake, and the end-to-end demo. The second is the cost. The baseline for cost is one of the other solutions, X25519MLKEM768 TLS 1.3, since it is the present adopted standard against quantum adversaries. The metrics are: Time To Last Byte (TTLB), handshake latency, the per-record key-consumption rate, the rate at which the KME pool drains under concurrent sessions, and the Cascade information leak as a function of the channel QBER. The goal is not to claim parity with the PQC baseline on raw throughput, but to quantify the price of unconditional security in a regime where it is the right price to pay.

The thesis makes three concrete contributions. (i) A reproducible end-to-end stack in which every primitive on the data path is information-theoretically secure, integrating a Qiskit-based BB84 simulation, a native Cascade reconciliation with back-propagation, an ETSI GS QKD 014 Key Management Entity, and a TLS 1.3 shaped protocol whose record layer uses One-Time Pad encryption and a Wegman-Carter MAC. (ii) An open-source reference implementation of the ETSI GS QKD 014 Key Management Entity (KME). It exposes the standard REST endpoints used by applications to fetch QKD keys, and includes a simulated link between the two KMEs at the endpoints of a QKD connection. (iii) An evaluation of the cost of unconditional security against the X25519MLKEM768 TLS 1.3 baseline, reporting Time To Last Byte and handshake latency over representative payloads, together with key-consumption and KME-pool-drain measurements that describe the behavior of such an implementation. One limitation has to be acknowledged up front: the privacy amplification step uses SHAKE-128 as its extractor rather than a two-universal hash family. The scope and impact of this caveat are discussed in the discussion chapter.

2. Background

Computing depends on communication between peers, and the problem it solves is old. Smoke signals were among the earliest forms of long-distance communication, and they already worked the way every protocol since has: the two sides agreed beforehand on what a given puff or sequence meant, and that shared convention, not the smoke, carried the message. Modern communication is the same idea grown to large scale. For two parties to interoperate they follow shared standards that fix how a message is framed, how a session opens, and what each side may assume about the other. The Transport Layer Security protocol, described later in this chapter, is one such standard, and it carries most of the encrypted traffic on the Internet.

Since the smoke, a lot has changed: communication has become an entire domain of its own, resting on a vast infrastructure that spans the planet and scales to almost any point on the globe. Most communication now happens between parties that have never met, across networks neither of them owns. A browser opens a session with a server it has no prior relationship with; a service calls another through infrastructure run by someone else entirely. The path a message takes is shared with unknown others, any of whom might read it or tamper with it along the way. A shared convention is no longer enough by itself. Agreeing

on what a message means does nothing to keep it private or intact while it crosses ground neither party trusts.

This is why in communication it matters how it is secured. Security is not one property but several guarantees at once: that an outsider cannot read the traffic, cannot alter it without being noticed, and is in fact the party it claims to be. These are confidentiality, integrity, and authentication, and a protocol like TLS exists to deliver all three over a channel it cannot trust. How each is achieved, and what it assumes, is the subject of the section on security models later in this chapter.

How strong those guarantees must be is not fixed. It depends on what the information is worth and how long it has to stay protected. A session token that expires in minutes only has to resist an attacker for minutes. A medical record or a state secret may need to stay confidential for decades, long enough that an adversary who just stores the traffic today and waits could still break it later. How much security is enough, and whether the strongest guarantee is even within reach, is itself part of the problem.

All of this rests on an assumption that is rarely contested. The key exchange that protects almost every connection on the Internet is only computationally secure: it holds because no known algorithm can recover the key in any feasible time, not because recovering it is impossible. The hardness it depends on is determined only by the absence of a fast enough attack so far.

This is a concern because the history of cryptography is full of algorithms that were trusted in their time and broken later, as computers grew faster and new algorithms were developed. The Data Encryption Standard (DES) algorithm was eventually broken by sheer brute force: in 1998 a purpose-built machine recovered a DES key in under three days [13], more than its 56-bit key could withstand. To continue on this idea, the hash function MD5, once used everywhere, was shown to admit practical collisions in 2004 [14], and SHA-1 followed in 2017 when the first full collision was produced [15]. In each case the algorithm was considered safe right up until it was proven faulty, and the data it had protected was exposed as a consequence.

To put this in perspective, there are two main problems in communication security. The first is that computational security is always provisional: it lasts only until someone finds a better attack or builds a faster machine. The second is worse. A large enough quantum computer would not chip away at today's key exchange but break it completely, solving in reasonable time the very problems it is built on. Even though such a quantum computer does not exist yet, an adversary can store encrypted traffic now and decrypt it once the capability exists, so data that must stay secret for decades is already at risk. This is what motivates the search for a guarantee that does not rest on any assumption about the attacker's resources, security that holds not because an attack is too expensive, but because it is impossible. Such a guarantee is called information-theoretic.

Alternative approaches and related work

There are two broad ways to defend communication against the quantum threat, and they differ at the root. The first keeps the architecture we already have and only replaces the broken

pieces: the vulnerable key exchange is swapped for one built on mathematical problems that no known quantum algorithm can solve. This is the idea behind post-quantum cryptography, treated in detail in the post-quantum cryptography section later in this chapter. It runs on ordinary hardware and fits inside existing protocols, which makes it the path the wider Internet is taking. Its security, however, is still computational: it rests on an assumption about how hard a problem is, a stronger assumption than before but an assumption all the same, and one that a future attack could in principle overturn just as earlier ones did.

The second way changes what the guarantee rests on. Instead of a problem that is merely hard to solve, it uses the laws of physics, which an adversary cannot outcompute no matter how much power it has. Quantum key distribution takes this route: two parties derive a shared key whose secrecy follows from the behaviour of quantum measurement rather than from any unproven hardness, as the quantum foundations and BB84 sections develop. The cost is that it needs dedicated quantum hardware and a physical channel between the parties, where post-quantum cryptography needs only software. This thesis follows the second route, using a QKD-derived key to drive an information-theoretically secure channel.

Integrating quantum-distributed keys into a standard secure-channel protocol has been attempted before. Rubio Garcia et al. [16] build quantum-resistant TLS by feeding a QKD-delivered key into the handshake, combining it with the negotiated secret so that the session survives even if the classical exchange is broken. Notably, they discuss using the QKD key as a one-time pad but set that option aside as too costly, and protect the data with a conventional cipher such as AES-GCM instead. The transport therefore stays computationally secure: the QKD key improves how the secret is established, but the data is still guarded by a computational primitive. This thesis takes the option they declined. The QKD key is not handed to a block cipher but consumed directly by a one-time pad and a Wegman-Carter authenticator, so the confidentiality and integrity of the data are themselves information-theoretic, and the cost of doing so is exactly what this work sets out to measure.

A related and now widely deployed line of work hardens the classical handshake without QKD at all. Hybrid key exchange, as specified for TLS 1.3 by Stebila et al. [17], runs a post-quantum KEM such as ML-KEM alongside the ordinary elliptic-curve exchange and derives the session secret from both, so the connection holds as long as either component resists attack. This is the migration path the wider Internet is taking, and it is pragmatic: it needs no new hardware and degrades gracefully. Its guarantee, however, is still computational on the data path, resting on the conjectured hardness of the lattice problem behind ML-KEM, detailed in the post-quantum cryptography section. This thesis takes the opposite position, keeping the data path purely information-theoretic and treating the computational assumptions as something to remove rather than to fall back on.

Goal and contributions

The question this thesis sets out to answer is whether a standard secure channel can be made *information-theoretically secure* end to end, using a key produced by quantum key distribution, and at what cost. Concretely: can the key-exchange and data-protection primitives of TLS 1.3 be replaced by ones whose security rests on physics and on Shannon's bounds rather

than on computational hardness, while keeping the surrounding protocol structure intact, and what price in throughput, latency, and key consumption does that replacement carry?

In answering it, this thesis makes the following contributions.

A working end-to-end implementation of QTLS-over-QKD, in which the TLS 1.3 architecture is preserved but its cryptographic core is replaced: the ECDHE key exchange by a QKD key fetched over the ETSI GS QKD 014 interface, the AES-GCM record protection by a one-time pad, and the HMAC authentication by a Wegman-Carter code over $\text{GF}(2^{128})$. The stack keeps the TLS data path information-theoretic rather than re-keying a classical cipher, the option that existing QKD-in-TLS work set aside as too costly.

A self-contained QKD layer: a BB84 simulation followed by full Cascade information reconciliation and privacy amplification, producing identical, secret keys at the two endpoints and feeding them into a single-pair ETSI 014 Key Management Entity.

An empirical evaluation of what unconditional security costs on the data path, measuring handshake latency, bulk throughput, and per-record key consumption against a classical TLS 1.3 baseline, and comparing the key-establishment step against both X25519 and ML-KEM-768.

An honest account of the boundary of the guarantee: a threat model that states plainly which parts are information-theoretic and which rest on assumptions (the authentication bootstrap and the trusted-node nature of the KMEs), so the claim is neither overstated nor hidden.

TLS 1.3 handshake

TLS 1.3 is the transport-layer protocol that secures the vast majority of encrypted traffic on the Internet today [1]. A TLS connection begins with a handshake whose purpose is to (i) agree on a shared secret between client and server, (ii) authenticate at least the server, and (iii) derive the symmetric keys that protect every subsequent application record. All later guarantees of the connection rest on this handshake.

The handshake is carried by a small fixed sequence of messages. The client opens with a *ClientHello* that mentions its supported cipher suites and, in its *key_share* extension, sends one or more ephemeral Diffie-Hellman public keys. The server replies with a *ServerHello* containing its own *key_share* at this point both peers can compute the shared secret and derive the keys that protect the rest of the handshake. Everything the server sends after *ServerHello* is already encrypted: *EncryptedExtensions*, optionally a *Certificate* and *CertificateVerify* (the server's signature over the transcript), and a *Finished* message that authenticates the handshake so far. The client answers with its own *Finished*, after which both sides switch to the application traffic keys.

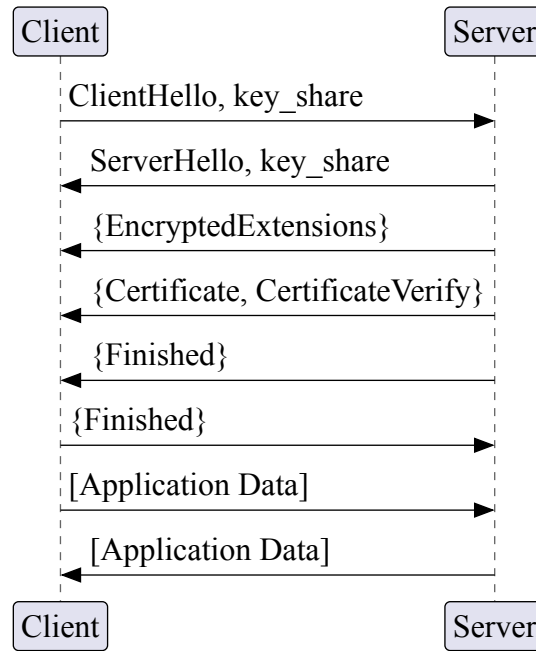


Figure 1. TLS 1.3 full handshake. Messages in braces {...} are encrypted under the handshake traffic keys; messages in brackets [...] under the application traffic keys.

The keys themselves are produced by a key schedule based on the HMAC-based key derivation function (HKDF) [18]. The shared secret from the key exchange is mixed through a sequence of *Extract* and *Expand* operations that create, successively, the handshake traffic secrets (protecting the rest of the handshake, the {...} messages) and the application traffic secrets (protecting the data phase, the [...] messages). The same schedule also produces the message authentication code (MAC) key used inside the *Finished* messages, which bind both peers to the full sequence of handshake messages they have exchanged and thereby detect any tampering by a man in the middle.

Two properties of this handshake matter for the rest of the chapter. First, the entire confidentiality of the connection depends on the secrecy of the value produced by the key exchange in the *key_share* extension: an adversary who can recover it can derive every traffic key that follows. Second, TLS 1.3 specifies the message flow and the key schedule, but the actual mechanism that produces the shared secret can be implemented in different variants. The next section makes precise what kind of security guarantee this primitive is expected to provide.

Security models and information-theoretic primitives

Cryptographic guarantees come in two strengths. A scheme is *computationally secure* if breaking it is possible in principle but requires an amount of computation that no realistic adversary can afford; its security rests on the assumed hardness of some problem, such as factoring or computing discrete logarithms. A scheme is *information-theoretically secure* (ITS) if it cannot be broken regardless of the adversary's computational power, because the ciphertext simply does not contain enough information to recover the plaintext. The distinction is vital for this thesis: a computational guarantee can be invalidated by a faster algorithm or a faster machine, whereas an information-theoretic one cannot.

Shannon formalised the notion of perfect secrecy [10]. A cipher achieves perfect secrecy if the probability of observing a given ciphertext c is the same whatever the underlying message m is:

$$\Pr[C = c \mid M = m] = \Pr[C = c] \quad \text{for all } m, c. \quad (1)$$

This is Shannon's necessary-and-sufficient condition: the ciphertext distribution is independent of the message, so observing c tells the adversary nothing it did not already know. Equivalently, in information-theoretic terms, the mutual information between message and ciphertext is zero,

$$I(M; C) = 0, \quad (2)$$

which is why the guarantee is called information-theoretic. Shannon also proved a price for it: perfect secrecy requires the key to be at least as long as the message and to be used only once.

The *one-time pad* (OTP) is the textbook cipher that attains this bound [19]. Given a message m and a uniformly random key k of the same length, used a single time, encryption and decryption are bitwise exclusive-or:

$$c = m \oplus k, \quad m = c \oplus k. \quad (3)$$

Because k is uniform and independent of m , every plaintext is equally consistent with a given c , so $I(M; C) = 0$. The OTP shows that perfect secrecy is achievable, but only at the cost Shannon identified: a fresh, truly random key as long as the data, never reused. Reusing any portion of the key is fatal for the secrecy.

Confidentiality is not integrity. An adversary who cannot read an OTP ciphertext can still flip chosen bits of the plaintext by flipping the corresponding bits of c . Protecting integrity with an information-theoretic guarantee requires a separate primitive: a *message authentication code* built from a family of hash functions, following Wegman and Carter [11], [20]. The idea is that the sender does not use a single, fixed hash function, but picks one at random from a whole family $\mathcal{H}$, and the secret key is exactly the choice of which function is used. Since the attacker does not know the key, it does not know which function was picked. The family is built so that any two different messages m and m' are mapped to the same tag by only a small fraction of its functions. Formally, a function h chosen at random from $\mathcal{H}$ produces a collision with probability at most ε :

$$\Pr_{h \in \mathcal{H}}[h(m) = h(m')] \leq \varepsilon. \quad (4)$$

The tag itself is not just $h(m)$: the hash value is hidden by adding a fresh one-time random string, the *mask*, so that the transmitted tag is $h(m) \oplus \text{mask}$. The mask stops the attacker from learning anything about which function was chosen from the tags it observes, just as the one-time pad hides the message.

To forge, an attacker who has seen one valid message and its tag would have to produce the correct tag for a different message. Because it does not know which function was used and collisions are rare, the best it can do is guess, succeeding with probability at most ε , regardless of how much computation it performs, making it information-theoretic [21]. The

cost, exactly as with the one-time pad, is that fresh, single-use key material (both the random choice of function and the mask) is needed for every message; reusing it breaks the guarantee.

Together, the one-time pad and a Wegman-Carter authenticator provide information-theoretic confidentiality and integrity, but both demand a continuous supply of uniformly random secret key shared by the two parties. Classical key exchange, the subject of the next section, does not provide such a supply unconditionally; it fails to provide more than computational secure keys.

Classical key exchange in TLS

The shared secret of the TLS handshake is produced by an authenticated Diffie-Hellman key exchange. In TLS 1.3 the only key-exchange family retained is *ephemeral elliptic-curve Diffie-Hellman* (ECDHE) [1]. Each peer sends a public value derived from a fresh secret in its *key_share*, and from its own secret and the other peer's public value both sides arrive at the same point on an elliptic curve, which is then hashed into the shared secret fed to the key schedule. The curves used are standardised, most commonly P-256 [4] or Curve25519. Because the secrets are *ephemeral* (the secrets are used once then discarded), the exchange gains a property called forward secrecy, meaning that, recovering one session's key does not compromise any other.

The security of this exchange is *computational*. An eavesdropper sees both public values but, to recover the shared secret, would have to solve the *elliptic-curve discrete logarithm problem* (ECDLP): given a base point G and a public multiple $Q = kG$ of it, find the scalar multiplier k [5],

$$\text{given } G \text{ and } Q = kG, \text{ find } k. \quad (5)$$

No efficient classical algorithm is known for ECDLP on well-chosen curves, and the best known attacks take time exponential in the key size, which is what makes ECDHE practical and secure against classical adversaries. This security, however, rests on the absence of an efficient algorithm, not on a proof that none can exist. Cryptography has a history of schemes once believed secure that were later broken when a better attack was found, and ECDLP is one of them: it is broken by Shor's algorithm on a quantum computer.

The quantum threat

In 1994 Shor discovered a quantum algorithm that solves both integer factorisation and the discrete logarithm problem in polynomial time [6], [22]. The same idea applies to the elliptic-curve discrete logarithm of Equation 5, so a sufficiently large quantum computer would recover the scalar k efficiently, and with it the shared secret of an ECDHE handshake. This means that the two hard problems underlying essentially all deployed public-key cryptography, factoring for RSA and discrete logarithms for Diffie-Hellman and its elliptic-curve variant, are broken by this algorithm.

The core of Shor's algorithm is *order finding*: the hardness of these problems can be reduced to finding the period of a function, and a quantum computer finds that period efficiently using the *quantum Fourier transform*. This works because a quantum computer can use superposition and interference to explore many states at once. What matters for this thesis is

the consequence: this algorithm is what forces the need for information-theoretic primitives in the face of a future quantum adversary.

It needs to be taken note that the algorithm exists but the hardware does not yet: running Shor against the curve sizes used in practice requires a fault-tolerant quantum computer with far more stable qubits than any built so far. Moreover, not all cryptography is equally affected. Grover’s algorithm gives only a quadratic speed-up for brute-force search [23], so primitives such as block ciphers and hash functions are weakened but not broken, and doubling the key length restores their security. The damage is concentrated on public-key key exchange, exactly the component that establishes the shared secret in TLS.

An adversary can record encrypted traffic today and decrypt it once a capable quantum computer becomes available, an attack known as *harvest now, decrypt later* [7]. Any data whose confidentiality must outlive the arrival of quantum computers is therefore already at risk, even though no such machine exists yet.

Post-quantum cryptography

The first response to the quantum threat is to keep the classical architecture but replace the broken primitives with new ones believed to resist quantum attack. This is the goal of *post-quantum cryptography* (PQC): public-key algorithms that run on ordinary computers yet rely on mathematical problems for which no efficient quantum algorithm is known [3]. The candidates are usually grouped by the hard problem they build on: lattice-based schemes (short vectors in a lattice), code-based schemes (decoding random linear codes), hash-based schemes (the security of a hash function, used mainly for signatures), and isogeny-based schemes (paths in graphs of elliptic curves). They differ widely in key size, speed, and maturity, and not all resisted attack: the isogeny key exchange SIKE was broken by a classical attack in 2022 [24], a proof that “post-quantum” means “no known quantum attack”, not “provably secure”.

The scheme standardised for key establishment is *ML-KEM* [8], a lattice-based key-encapsulation mechanism derived from CRYSTALS-Kyber. Its security rests on the *learning with errors* (LWE) problem. LWE asks one to recover a secret vector s from many noisy linear equations: one is given pairs (a_i, b_i) with

$$b_i = \langle a_i, s \rangle + e_i, \quad (6)$$

where the a_i are random, $\langle \cdot, \cdot \rangle$ is the inner product, and each e_i is a small random error. Without the errors this is ordinary linear algebra and s is trivially recovered; the small noise terms e_i are what make the problem hard, and no efficient algorithm, classical or quantum, is known to solve it for suitable parameters.

In practice PQC is not deployed alone but in a *hybrid* key exchange: the TLS handshake runs both a classical primitive (ECDHE) and a post-quantum one (ML-KEM) in parallel, and the shared secret is derived from both. This is the migration path currently being adopted on the Web.

ML-KEM is secure because no efficient algorithm for MLWE is known, exactly the kind of guarantee that Shor’s algorithm overturned for ECDLP.

Quantum foundations

The security of the key exchange in the next section rests on two physical facts about quantum measurement. This section establishes both. The physical carrier of quantum key distribution is the *qubit*, a two-level quantum system whose state is a superposition of two basis states [12].

The *computational* (or Z) basis is $|0\rangle$ and $|1\rangle$; the *diagonal* (or X) basis is

$$|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle), \quad |-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle). \quad (7)$$

The two bases are *conjugate*: a state that is definite in one basis is uncertain in the other. Take $|+\rangle$. It is perfectly defined in the X basis, yet in the Z basis it is an equal superposition of $|0\rangle$ and $|1\rangle$.

In order to read a qubit we need to measure it, and measuring destroys the state. A measurement in a given basis projects the qubit onto one of that basis's two outcomes and returns the matching bit. If the qubit was prepared in the same basis, the result is deterministic and the state survives: measuring $|0\rangle$ in the Z basis gives 0 with certainty. If it was prepared in the *conjugate* basis, the outcome is uniformly random and the original state is gone: measuring $|+\rangle$ in the Z basis gives 0 or 1 with equal probability, and the qubit is left in $|0\rangle$ or $|1\rangle$ accordingly. An observer who does not know the preparation basis cannot read the bit without risking both an error and a disturbance to the state.

The second physical fact is the *no-cloning theorem*, which makes the same point another way: no quantum operation copies an arbitrary unknown state [12]. The argument is a proof by contradiction. Suppose such an operation U existed, satisfying $U(|\psi\rangle|0\rangle) = |\psi\rangle|\psi\rangle$ for every state $|\psi\rangle$. Apply it to two different states $|\psi\rangle$ and $|\varphi\rangle$. Quantum operations preserve inner products, so equating the inner product before and after gives $\langle\psi|\varphi\rangle = \langle\psi|\varphi\rangle^2$. This holds only when $\langle\psi|\varphi\rangle$ is 0 or 1, that is, only when the two states are orthogonal or identical. A U that clones arbitrary states would have to satisfy it for every pair, which is a contradiction, so no such operation exists. An eavesdropper therefore cannot duplicate a qubit in flight, measure one copy, and forward the other untouched.

Both facts rest on physics rather than on the hardness of a computation, so the protection they give against eavesdropping holds regardless of the adversary's computing power.

The BB84 protocol

BB84, due to Bennett and Brassard [12], [25], turns the facts of the previous section into a way for two parties, Alice and Bob, to agree on a shared random string that an eavesdropper cannot learn without being detected. For each bit, Alice picks a random bit value and a random basis, Z or X, encodes the bit in that basis (a 0 as $|0\rangle$ or $|+\rangle$, a 1 as $|1\rangle$ or $|-\rangle$) and sends the qubit to Bob. Bob, not knowing Alice's basis, measures each qubit in a basis he also chooses at random. If their bases coincide, his result equals Alice's bit; if they differ, his result is random and carries no information.

After transmission the two perform *sifting* over an authenticated public channel: they announce the basis used for each qubit, without revealing the bit values, and discard every

position where their bases differed. On the remaining positions, where the bases matched, their bits agree in the absence of noise and eavesdropping, and these form the *sifted key*. Since the bases are independent and uniform, on average half the qubits survive sifting. Table 1 illustrates one round.

Table 1. One round of BB84. Positions where the bases differ are discarded; the matching positions form the sifted key.

Alice bit	1	0	1	1	0	0
Alice basis	Z	X	Z	X	Z	X
Bob basis	Z	Z	Z	X	X	X
Bases match	yes	no	yes	yes	no	yes
Sifted key	1		1	1		0

```

function bb84_round(n):
    // sifted holds the positions where both sides chose the same
    basis
    sifted <- empty list
    for i = 1 .. n:
        alice_bit    <- random bit
        alice_basis   <- random basis (Z or X)
        bob_basis     <- random basis (Z or X)
        qubit        <- encode(alice_bit, alice_basis)    // quantum
    channel
        bob_bit      <- measure(qubit, bob_basis)
        // sifting, over the authenticated public channel:
        if alice_basis = bob_basis:    // bases announced, not
    bit values
        append (alice_bit, bob_bit) to sifted
    return sifted

```

Listing 1: One round of BB84: per-qubit preparation and measurement, followed by sifting on the positions where the two bases agree.

But realistically, the sifted key is not yet usable. Channel noise and any eavesdropping introduce disagreements between Alice’s and Bob’s versions, measured by the *quantum bit error rate* (QBER), the fraction of sifted positions where their bits differ. Alice and Bob estimate it by publicly comparing the bits on a random sample of the sifted key and then discarding that sample. By the disturbance property, an eavesdropper who measures qubits in a basis she has guessed wrong corrupts them and so raises the QBER; an observed error rate above a threshold, derived from the Shor-Preskill security proof of BB84 [22], signals interception and the key is aborted. A low QBER bounds how much information an eavesdropper can have obtained.

In the case the QBER is below that threshold, the remaining errors are removed by *information reconciliation*, a public-discussion procedure that lets Alice and Bob correct the differing bits and reach an identical string. The implementation chosen for this work uses Cascade [26], [27], which proceeds in passes: the key is split into blocks, parities of the blocks are

compared, and a binary search inside any block with mismatched parity locates and flips the problem bit, with later passes catching errors the earlier ones missed. Reconciliation leaks some information to the eavesdropper, because the exchanged parities are public.

Two adjustments then turn the reconciled string into a secure key. First, the information leaked during reconciliation, together with whatever the eavesdropper may have gathered from the channel within the QBER bound, is removed by *privacy amplification* [28]: Alice and Bob apply a randomly chosen function from a universal family (the universal families of the security-models section) to compress their shared string into a shorter one about which the eavesdropper's information is negligible. The leftover hash lemma [29] quantifies how much the string must be shortened as a function of the eavesdropper's estimated knowledge. The result is a shared key whose secrecy rests on the laws of physics rather than on any computational assumption.

The ETSI QKD 014 key delivery standard

ETSI GS QKD 014 standardises the interface through which an application requests keys from QKD hardware [9]. It separates the system into two roles: the *Key Management Entity* (KME), attached to a QKD device and storing the keys it produces, and the *Secure Application Entity* (SAE), the application that consumes them. This way, the TLS stack is an SAE and never speaks to the quantum hardware directly, but asks its local KME for keys over a defined API. Each KME serves the SAEs at its own site over a local, secured channel.

A pair of SAEs obtains a shared key as follows. The master SAE asks its local KME for key material; the KME returns one or more keys, each paired with a *key_ID*. The master SAE passes the *key_ID* to the slave SAE over its own application channel, and the slave SAE presents that *key_ID* to its own KME, which returns the identical key. The two SAEs then hold the same secret without ever exchanging it directly. The standard defines this through a small REST API with three endpoints: */status*, which reports the key material a KME has available for a given peer; */enc_keys*, by which the master SAE requests new keys and receives them with their *key_ID*s; and */dec_keys*, by which the slave SAE retrieves a key by its *key_ID*.

This model carries a strong assumption: the two KMEs hold the keys in the clear and exchange them between sites, meaning the security of the delivered key depends on each KME being uncompromised (in QKD terminology each KME is a *trusted node*). The quantum guarantee covers the link between the QKD network, but the KMEs that store and forward the keys are trusted by assumption, not protected by physics.

Threat model

The foundations above can now be combined into the adversary against which the rest of the thesis reasons. The adversary is assumed to be computationally unbounded in the quantum sense: it has access to a large-scale quantum computer and can run Shor's algorithm. It can read, store, and tamper with all classical traffic, and in particular it can mount the harvest-now-decrypt-later attack, recording today's traffic to break it later. On the quantum channel it can intercept and tamper with qubits in transit, but it cannot clone an unknown state or read a qubit without disturbing it, by the facts of the quantum foundations section.

Against this adversary the stack splits cleanly into two layers. The key itself is *information-theoretically secure*: it is produced by BB84, whose secrecy follows from physics and whose residual leakage is removed by privacy amplification, and once installed it protects data through a one-time pad and a Wegman-Carter authenticator, both unconditionally secure. The classical key exchange of TLS, by contrast, would be broken by such an adversary, which is precisely why it is replaced.

Two assumptions remain outside the information-theoretic guarantee. First, authentication of the public channels, both the sifting and reconciliation discussion and the SAE-to-SAE exchange, relies on the parties already sharing a short secret to key the Wegman-Carter authenticator. Second, the trusted-node assumption of the previous section: the KMEs hold key material in the clear, so an adversary who compromises a KME obtains the keys directly. The information-theoretic claim of this work therefore applies to the key agreement and the data path given uncompromised.

3. Architecture

Solution overview

The solution is a secure communication system for parties who need their traffic to stay secret no matter how much computing power an attacker has. The way it works is that the two communicating endpoints obtain a shared secret without a public-key exchange: instead of negotiating a key over the network, both sides draw identical key material from a quantum key distribution (QKD) link and use it to protect their traffic. Once both sides hold the same key, the endpoints run a TLS-style handshake to authenticate the channel and then encrypt the application payload.

We describe the architecture abstractly as a small set of replaceable black boxes. The system has two endpoints. Each end holds a QKD endpoint, a KME that mediates access to the keys, and a SAE that consumes them. The two QKD endpoints face each other over the quantum link and converge on the same key; the two KMEs hand that key to their local SAE over a standard interface, and the two SAEs run the secure session between them. Two other boxes observe the system, one to evaluate it and one to monitor it. Every box can be replaced with different hardware or algorithms, as long as the system still delivers information-theoretically secure ciphertext. The QKD link is a good example: it cannot be replaced with a classic channel, because that would defeat the purpose of the thesis.

Application stack

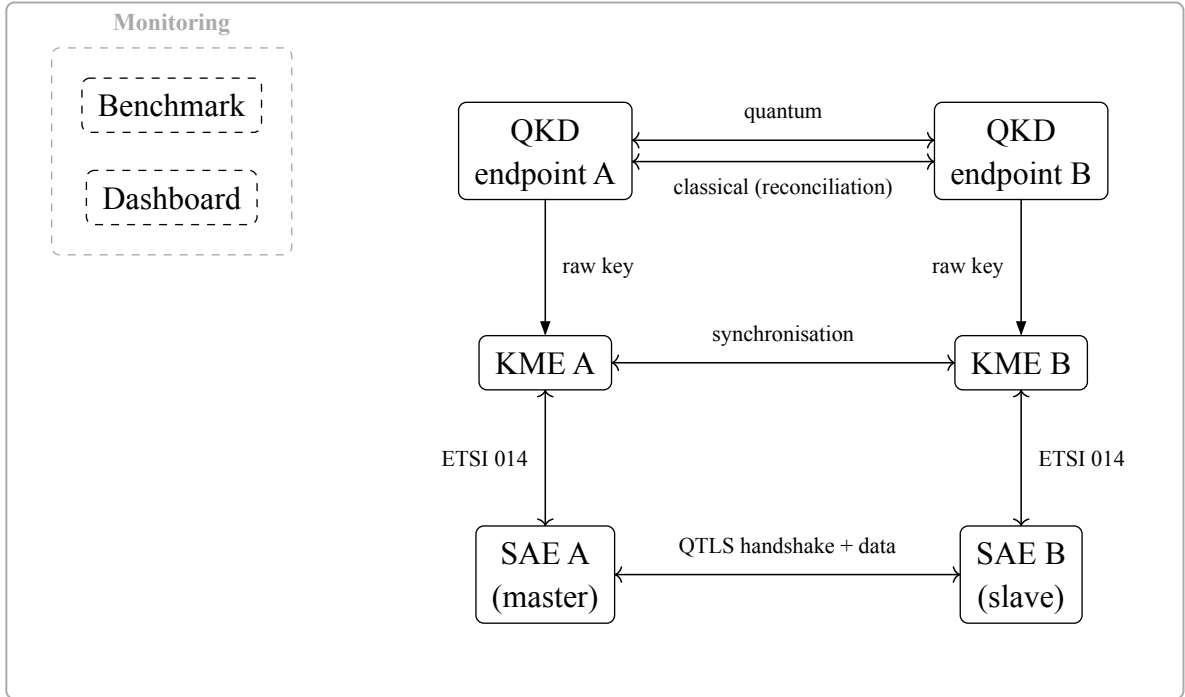


Figure 2. System architecture. Each end holds a QKD endpoint, a KME and a SAE. The five protocol channels are: QKD to QKD (a quantum link and a classical public link), QKD to KME (key production), KME to KME (synchronisation), SAE to KME (ETSI 014), and SAE to SAE (the QTLS data path). The benchmark and dashboard monitor the running stack.

Functional decomposition

This section breaks the system into its functional blocks. For each block we state what it is responsible for and what it exposes to the others, but not how it works inside; the internal algorithms are left for the implementation chapter.

Secure application entity (SAE)

The SAE is the application-facing end of the system. Its point is to send data securely to the other endpoint, and it hides everything about QKD behind a familiar connection interface. An SAE does two things. First, it obtains a key from its local KME over the ETSI interface: the client SAE pulls a fresh key and gets back a key identifier together with the key, and the server SAE later resolves that same identifier into the same key. Second, it runs a TLS-style handshake with the SAE on the other end, which carries the key identifier as an authentication mechanism to prove that both sides hold the same QKD key without ever sending the key over the wire. After that, the rest of the communication is secured using the QKD keys.

The SAE has three interfaces. Towards the application it exposes a secure connection: one side opens it as a client and the other accepts it as a server, after which both can send and receive data and finally close the connection, much like an ordinary secure socket. Towards the KME, the SAE is only a client of the ETSI interface, meaning it requests and resolves keys by their identifier and exposes nothing back. Towards the peer SAE, it exposes the handshake

messages that carry the key identifier and prove key possession, followed by the protected data stream. The SAEs work asymmetrically: one acts as the master that pulls a fresh key and one as the slave that resolves the identifier it is given. Once the session is established the roles no longer matter and the two ends are interchangeable.

Key management entity (KME)

The KME sits between the QKD endpoint and the SAE. The QKD endpoint produces a stream of raw key material but knows nothing about who wants it; the KME turns that stream into a service an application can actually use. In practice, it stores the material it receives, gives each key an identifier, and hands keys out on request, while making sure a key is never given to an SAE that is not allowed to have it and never handed out twice. This way, the SAE never touches the QKD endpoint directly.

A KME talks over three interfaces. Towards its local QKD endpoint it only receives: the endpoint pushes finished key material into the store, and the KME accepts and keeps it. Towards its local SAEs it exposes the ETSI interface, which behaves differently depending on the type of SAE: a master SAE asks for a fresh key and receives an identifier together with the key, and a slave SAE asks for the key behind a given identifier. On every such request the KME also checks that the requesting SAE is authorised. Towards the other KME it exposes a synchronisation interface. The two KMEs must give the same key material the same identifier on both ends, so that a key the master pulls on one side resolves to the same key on the other, and they must ensure that a key reserved on one side is never reused on the other. The KME exposes the ETSI interface to its SAEs and the synchronisation interface to its peer, but it never exposes the QKD endpoint to anyone.

QKD endpoint

The QKD endpoint is the provider of key material. The way it works is: there is one at each end, and the two of them are what actually let the system work: through the laws of quantum mechanics they arrive at the same secret key in two separate places, without that key ever travelling over a channel an attacker could tamper with.

A QKD endpoint has three interfaces. With the QKD endpoint at the other end it shares a quantum channel, over which the raw key is established, and a public classical channel, over which the two ends compare notes and reconcile the errors the quantum channel introduced. Towards its local KME it pushes the finished key material. The result of that exchange is a block of final, agreed key material that is identical on both ends. Towards its local KME, the endpoint only pushes information. How the key is actually produced and corrected is internal to this box and is described in the background and implementation chapters.

Benchmark module

The benchmark module is used to observe the metrics in the system. Its job is to measure the system under controlled conditions and produce the data the evaluation chapter reports on: how long a handshake takes, how much application data the one-time-pad path sustains, the time-to-last-byte (TTLB) [30], how fast the key pool drains under load, how the key rate behaves as the error rate changes, and how the key exchange compares against classical and post-quantum alternatives. It drives a full instance of the entire stack and runs a chosen set of scenarios and writes the raw results and the resulting plots to disk, so that a run is reproducible and the output can be regenerated without re-measuring.

Visualisation dashboard

The second observing box is a live dashboard that shows how the implementation works in real time: the key blocks as they are produced, the handshake as it advances step by step, the records as they are sent, and the key pool as it fills and drains. It attaches to a running instance of the stack and listens to the events each layer emits, without influencing the protocol. Its purpose is to give the system a visual demo, useful both during development and for showing a live demo. It exposes a browser interface a person can watch, and consumes the events the running stack publishes.

Interfaces and communication

The communication between the boxes is done in five different communication channels. The first channel is the one between the two QKDs that implements both a quantum and classical link for reconciliation. The quantum link is used for sending the qubits that create the quantum keys. The second channel is between the two different KMEs and it uses a link for synchronisation between the two ends, making sure we do not use the same key in two different records and that the keys used by both endpoints are corresponding. The third channel is between the QKD and the KME, being a one way channel, the KME consuming the raw produced keys from the QKD anytime it sends them. The fourth channel is between the KME and SAE, using the ETSI 014 format, and its responsibility is to make sure that the SAEs get the keys from the KMEs so they can secure their data. The last channel, between SAEs, is used for data communication and initiating the TLS-style handshake to authenticate.

The flow of a key used in securing a record is as follows: first, the two QKD nodes each arrive at the same key, which is reconciled over the classical link, and then it is given to the KMEs. The KMEs parse the raw key output and generate an identifier that they have to synchronise on both ends, so the same key does not get a different identifier on the two KMEs, which would make it unusable. After the synchronisation process is done, the key is ready to be used by the SAEs in two different scenarios. If SAE A is the one that initiated the connection (the master), it will request the key and its id from its KME A, then the master SAE initiates a TLS-style handshake giving the slave SAE the key id so it knows what key to request from its KME. After that, both SAEs have the same key and can secure their connection using the same flow, which can be interchanged, meaning the SAEs can be both slave and master in the same connection. At the same time the KMEs make sure that they stay synchronised and keep track of which keys were used and which keys are available.

Once the session is running, the two SAEs no longer talk to the KMEs. Each record consumes a slice of the shared key, and both sides advance through the same key, so they stay in sync on their own.

Each channel has to respect some rules so the boxes can be replaced without breaking the others. The channel between the QKD and the KME only has to deliver finished key material, so the KME does not care how the key was produced. The channel between the two KMEs has to guarantee that the same key gets the same identifier on both ends and that a key is never handed out twice. The channel between the KME and the SAE has to follow the ETSI 014 format, so any SAE that speaks in that standard can request keys. The channel between

the two SAEs has to follow the handshake protocol, so the key id is carried across and both sides prove they hold the same key before any data is sent.

Non-functional considerations

The architecture supports modularity (boxes can be replaced with different implementations as long as they respect their set of rules), encapsulation (the KME does not need to know how the QKD works) and separation of concerns (the KME is used for key preparation, delivery and synchronisation, the QKD is used to generate key material and the SAE is the consumer and maintainer of the connection). For a common consumer of the stack, the implementation is abstracted so the client needs only to understand the ETSI 014 standard to receive the keys and the handshake protocol.

In addition, the communication between SAE and KME is standardized (ETSI 014), making it easy to implement, and the data path between the two SAEs is information-theoretically secure at a cost (OTP encrypts the communication with keys as long as the message), while other channels (authentication from SAE to KME, QKD to QKD classic link for reconciliation etc.) depend on infrastructure. The TLS handshake has overhead caused by requesting the keys from its KME through the ETSI 014 API and a session is initiated by a handshake and ends when the SAEs consume the key they requested from the KME, highlighting that the stack uses QKD keys. For evaluation, the stack gives transparent data evaluation and visualisation that can be reproduced.

By design, a session stops once its key runs out, instead of quietly fetching more. Because a one-time pad spends one key byte per data byte, this makes the cost of OTP and the QKD key rate visible. The alternative would be to fetch a new key mid-session and switch to it, which is an extension left for future work.

Mapping to implementation and evaluation

Having the architecture presented, in order to implement the stack there are a lot of decisions that need to be taken and make sense with the proposed layout. The SAE becomes the QTLS endpoint, which integrates the handshake behaviour, the key schedule, and the one-time-pad and Wegman-Carter record protection, together with the client that talks to the KME. The KME becomes the key store, the ETSI 014 REST API, and the synchronisation path between the two stores. The QKD endpoint develops the key generation, the reconciliation and privacy-amplification. The benchmark module and the dashboard become the measurement runner and the live demo visualisation.

The evaluation chapter then validates these boxes through the experiments the benchmark module runs: the handshake latency and time-to-last-byte against the post-quantum TLS baseline, the rate at which the key pool drains under concurrent sessions, the per-record key consumption, and the reconciliation leakage.

4. Implementation

Implementation overview

The whole stack is implemented in Python and its libraries, with a single small component in C. Python's well-developed libraries, together with it being an easy language to understand

(which makes it easier to concentrate on the critical parts of the application), made it the right choice for a research prototype whose goal is to show that the protocol works and to measure its cost. The language has strong cryptographic library support, supports asynchronous I/O, and lets the QKD layer, the KME, the QTLS endpoint, the benchmark harness and the dashboard all live in one process and one event loop which keeps the demo in a single observable place. The one place where Python was not enough is the Cascade reconciliation, which is written in C and called through `ctypes`.

How the stack is assembled

Although the architecture describes two ends, each with its own QKD endpoint, KME and SAE, the implementation runs all of them inside a single process and a single `asyncio` event loop [31]. This way, every layer the thesis describes is actually running and exercising the same code the tests cover, rather than being mocked. A small orchestrator owns every background task and wires the layers together as an asynchronous context manager, so that the whole system can be run or stopped.

The flow is built bottom-up, meaning that a producer generates final key blocks and puts them on a shared queue. A feeder then takes each block off the queue and writes it into both key stores, KME A and KME B, under the same identifier, which is what keeps the two stores synchronised. The orchestrator waits until both stores hold at least one key before it lets any handshake start, so the first connection cannot race ahead of the producer.

After the KMEs hold at least one key each, the SAEs begin the TLS-style handshake. Each one reaches its KME without going through the network: instead of opening a real socket, the requests are delivered straight into the KME application through its ASGI (Asynchronous Server Gateway Interface) interface, the standard interface a Python web server uses to talk to a web application. The full ETSI 014 request path still runs, only the network transport is skipped. The QTLS (SAEs) server and client, by contrast, talk to each other over a real loopback TCP socket. The master SAE pulls a key, the handshake carries its identifier to the slave, the slave resolves the same key from its own KME, and from there the two SAEs exchange protected records until the key runs out.

The orchestrator only assembles components that already exist and are tested on their own. This way, what runs in the live demo is exactly the code validated by the earlier tests, with the orchestrator adding only the wiring.

Technology choices

The QKD layer has been implemented in two ways that are interchangeable sources, both of which produce the same kind of final key block. The first is a simulation built on Qiskit [32]: one round of BB84 is expressed as a real quantum circuit and run on the AerSimulator using the stabilizer method, which handles many qubits per round almost instantly because BB84 only uses Clifford gates. Figure 3 shows one such round for four qubits, covering every combination of preparation and measurement basis.

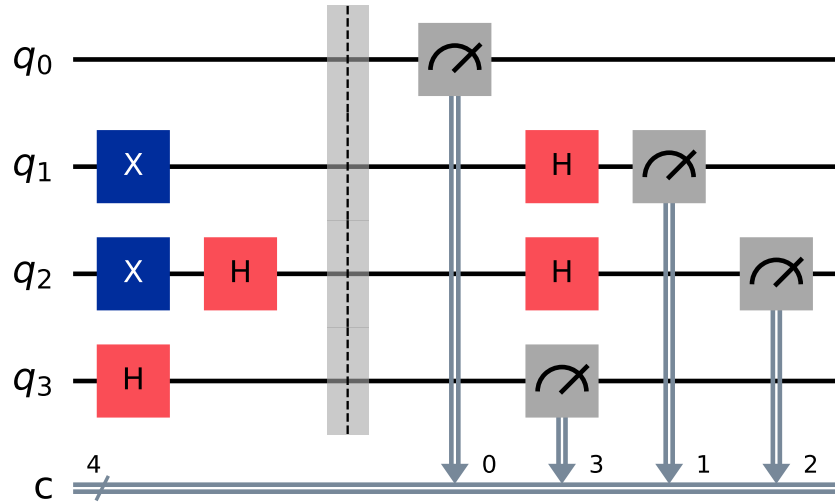


Figure 3. A BB84 round as a Qiskit circuit on four qubits. Alice’s preparation (the leftmost gates) and Bob’s measurement bases together cover every combination of the Z and X basis.

The second is an abstract key generator that does not simulate qubits, it draws random key material, applies a configurable error rate to model a noisy channel, and runs that through the same reconciliation and privacy amplification as BB84. This is the source that feeds the stack and the benchmarks. Using an abstraction is needed because simulating bb84 rounds and reconciliation would need strong computational power.

The KME exposes its ETSI 014 interface with FastAPI [33]. ETSI 014 is a REST interface with strict request and response formats, and FastAPI maps onto it directly: the data formats become Pydantic [34] models that validate every field against the standard, and the four endpoints become route handlers. A custom HTTP server written from scratch was rejected because it would mean reimplementing validation and routing for no protocol gain.

The SAE talks to the KME with httpx [35], chosen over the more common requests library for two reasons: it is asynchronous, which fits the single event loop design, and it can drive a FastAPI application in process through its ASGI interface, so the tests exercise the full ETSI request path without opening a real socket. A socket connection works with this implementation too.

Reconciliation runs the Cascade algorithm [26], implemented in C language and loaded through `ctypes`. Cascade is the bottleneck of the key procedure because it makes many passes over the raw key, and a pure Python version was too slow to keep the key rate believable and significant. In case the C implementation cannot be run, the code will show an error and stop itself.

The cryptographic primitives that are not information-theoretic, the HKDF key schedule, the SHA-2 transcript hashing, and the X.509 certificate handling for mTLS, come from the cryptography [36] library rather than being written by hand. The one-time pad and the

Wegman-Carter MAC, which are the information-theoretic core, are implemented directly so their cost and behaviour are fully visible in the measurements.

The dashboard is a Streamlit [37] application with Plotly [38] charts, chosen because it turns a single Python script into a live browser dashboard with almost no interface code, which is not the scope of this thesis. The benchmark suite uses pyperf [39] for the measurements that need a stable average. The handshake is timed directly instead, because there the interesting number is the real cost of a single reconnection, not an average over many runs.

Core components

This section describes how the central components are built. Each one is presented together with the problems and design decisions it raised. They are considered to be critical parts of the stack and went through several iterations before reaching their current form.

Key management entity

Each KME holds its keys in a store indexed by identifier. When a master SAE requests a key, the store checks the access list, takes the oldest unused key, marks it reserved for that master and slave pair, and mirrors the reservation to the peer store (the slave KME) before returning. The mirror happens while the shared lock is still held, so the local mark and the peer reservation commit as one step, which is what stops the same key from being delivered on both ends.

```
function take_key(master, slave):
    acquire shared lock
    if not authorised(master, slave):
        abort: access denied
    key <- oldest unused key in store
    if no such key:
        abort: pool empty
    mark key reserved for (master, slave)           // local
    mirror reservation (key.id, master, slave) to peer // same lock
    release shared lock
    return (key.id, key.material)
```

Listing 2: Reserving a key on the master KME. The local mark and the peer mirror commit together under one lock, so a key is never delivered twice.

KME-to-KME synchronisation

The two KMEs that an ETSI 014 deployment places at the ends of a QKD link must agree atomically on which key material backs which `key_ID` and on the SAE pair each delivered key is reserved for. In a distributed deployment this is done over an authenticated KME-to-KME channel, typically a two-phase commit, so that a master pull and the corresponding peer reservation either both succeed or both fail.

The implementation reached its current form through two iterations. The first design gave each `KeyStore` its own lock and turned out to be both slow, because every reservation paid a lock hand-off between the two stores, and racy, because a master could pull a key on KME A before the feeder had finished propagating the same block to KME B. The final design replaces the two locks with a single reentrant lock (`threading.RLock`) shared by both stores through `link_peer`, so that marking a key locally and mirroring the reservation

to the peer happen under one lock and commit as one atomic step. The same lock also guards `add_block` in the feeder, closing the race. The lock is reentrant because `take_keys` calls `peer.apply_reservation` while still holding it, and `apply_reservation` re-acquires the same shared lock. Using a reentrant lock lets the holding thread re-enter instead of deadlocking, so each function that needs atomicity stays correct on its own, with its atomicity guaranteed by the lock it takes rather than by which caller invoked it. Microbenchmarks confirmed that at the demo’s concurrency level this shared lock is not a bottleneck.

This is a simplified synchronisation implementation that works and lets off some complexity. A production KME would replace the shared lock with a proper distributed protocol such as two-phase commit, but the rest of the storage and API code would be unaffected by that substitution.

Scope of the ETSI 014 implementation

The KME component implements the **single-pair** subset of ETSI GS QKD 014 V1.1.1: one master SAE, one slave SAE, and one peer KME on the other end of the QKD link. Multicast key delivery to multiple slave SAEs (the optional `additional_slave_SAE_IDs` field of the Key request data format in clause 6.2 of the standard) is out of scope, as is the vendor and forward-compatibility extension surface (`extension_mandatory`, `extension_optional`, `status_extension`, `key_extension`, `key_ID_extension`, `key_container_extension`). The endpoints that the standard prescribes [9] are exposed in full (`GET /status`, `GET/POST /enc_keys`, `GET/POST /dec_keys`), with the request and response data formats matching the standard verbatim for every field that lies inside that single-pair scope. An example of a status response carries `source_KME_ID`, `target_KME_ID`, `key_size` and `stored_key_count`, and a delivered key is a `key_ID` paired with its Base64-encoded `key` material. Each format is a Pydantic model, so a request or response that does not match the standard’s shape is rejected before it reaches any protocol logic.

Not implementing the forward forward-compatibility and extension functionalities was done in favor to not make the thesis add a substantial amount of KME-to-KME coordination logic without changing any of the properties the thesis evaluates: key-ID synchronisation, no-reuse, ETSI wire-format conformance, and the cost of staying information-theoretically secure on the application data path. In case this functionality wants to be developed it would require to extend the existing `KeyStore` mirror mechanism rather than restructure it.

Authentication and the mTLS gap

ETSI 014 clause 5.1 [9] states that “all communications between SAE and KME shall use the HTTPS protocols (with TLS version 1.2 or higher)” and that “at the connection establishment, mutual authentication between SAE and KME shall be performed”, with the SAE ID derived from the Common Name (CN) of the client certificate.

The cryptographic simulation for that mTLS is built in the stack (a self-signed demonstration Certificate Authority together with X.509 leaf-certificate issuance, where the SAE ID is carried in the leaf CN, as the standard prescribes), and its end-to-end validity is exercised by an integration test (`test_mtls_cert_chain_round_trip`) that verifies a client certificate chains back to the demo CA and that its CN is the expected SAE ID.

At runtime, the SAE identity is resolved through a single injectable dependency, `current_sae`, that reads an `X-SAE-ID` header instead of the certificate's CN. The assumption behind this is that, for what the thesis measures, how an SAE is identified is not critical and the key-ID synchronisation, no-reuse, and ETSI wire-format conformance are all determined by the key store and the route handlers, none of which depend on the identification method. Authentication here is a property of the surrounding infrastructure, so we simplify it to a header and keep identity resolution in one place. A upgrade would be implemented by serving the KME over real sockets with mTLS and extracting the CN from the verified peer certificate and because `current_sae` is the only place identity is resolved, that upgrade touches no other module.

Extensibility

To support extensibility, the KME data structures are organised so that the extension fields the standard reserves “for future use” can be added without touching the storage layer or the KME-to-KME synchronisation path. The `KeyStore` indexes blocks by their UUID and treats the rest of each block as opaque payload; adding a `key_extension` object would amount to a new optional Pydantic field on the `Key` model and a pass-through in the API serialiser. The same is true of `status_extension` and the request-side `extension_mandatory / extension_optional` arrays: parsing them is local to the request handler, and the storage contract is unchanged.

QKD layer

After sifting, Alice's and Bob's keys still differ on a fraction of positions given by the QBER. The reconciliation step first checks that the QBER is below a configurable abort threshold, the Shor-Preiskill gate, and raises if it is not, so a channel that is too noisy or under eavesdropping never yields a key. Below the threshold, it runs Cascade to correct the differing bits and counts exactly how many parity bits were revealed on the public channel. Privacy amplification then shrinks the corrected key by at least that many bits, using a hash that removes whatever an eavesdropper could have learned, so the final key is bit-identical for both ends and secret. If Cascade fails to converge, the result is treated as an abort rather than a bad key. Listing 3 sketches the algorithm.

```

function reconcile_and_amplify(alice, bob, qber):
    if qber >= abort_threshold:           // Shor-Preskill gate
        abort: QBER too high

    // Cascade reconciliation, over the public channel
    leaked <- 0
    for pass in 1 .. P:
        shuffle both keys by an agreed permutation
        split into blocks
        for each block:
            if parity(alice_block) != parity(bob_block):
                binary-search the block to flip one differing bit
                leaked <- leaked + bits announced
            revisit earlier passes for newly-exposed odd blocks
    if alice != bob:
        abort: failed to converge

    // Privacy amplification
    shrink <- leaked + eve_information      // bits to remove
    secure_length <- len(alice) - shrink
    key <- SHAKE128(alice).output(secure_length) // variable-length
    hash
    return key // identical on both ends, and secret

```

Listing 3: The reconciliation and privacy-amplification pipeline. Cascade corrects the differing bits while tallying the parity bits it reveals; privacy amplification then removes at least that many bits.

QTLS endpoint

The SAE is the component an application actually uses to send data and it ties together the KME client and the secure session. Each SAE owns a client to its local KME, through which it requests or resolves keys by identifier, and it speaks the QTLS protocol to the SAE at the other end. To the application above it, the SAE looks like an ordinary secure socket: one side calls `connect` and the other `accept` to establish the session, after which both use `send`, `recv` and `close`. The two endpoints are asymmetric only while the key is obtained, the master pulls a fresh key and the slave resolves it by identifier, but once the session is up the two directions are independent and the roles no longer matter.

The protocol is best understood as TLS 1.3 with the (EC)DHE exchange replaced. QTLS removes that step and substitutes the QKD key looked up from the KME. Everything around it follows RFC 8446: the same message flow, the same transcript handling, the same Finished step. This makes it still be TLS like and it is also where its guarantee changes, since the shared secret no longer comes from a computational assumption but from the quantum channel.

That QKD key feeds a key schedule that follows the structure of RFC 8446 section 7.1, simplified to the secrets this protocol actually uses: a single `HKDF-Extract` from the QKD key, where TLS uses the (EC)DHE secret, with no pre shared key or early secret stage, followed by the same `Derive-Secret` and `HKDF-Expand-Label` steps and labels. From it the schedule derives the traffic keys for each direction, the material for the Finished messages, and the one-time-pad stream for the records.

The handshake itself is a short state machine: the client sends its key identifier in the ClientHello, checks that the ServerHello echoes the same identifier, and the two sides exchange Finished messages. Authentication is where QTLS departs most visibly from TLS, as there are no certificates or signatures: each side proves its identity simply by holding the same QKD key, demonstrated by the Finished message, a Wegman-Carter tag over the transcript that also proves the transcript was not tampered with. These messages are serialised as compact JSON rather than TLS's byte-exact binary, which adds parsing code but no security and keeps the handshake readable in captures.

Once the handshake completes, the data path is protected with the information-theoretic primitives introduced in the background chapter. Confidentiality comes from a one-time pad: the payload is XOR-ed with key bytes drawn from the key stream, which holds the QKD material behind an offset that only moves forward, so a byte is never reused and a request for more than remains raises rather than wrapping, turning key exhaustion into a hard stop. Integrity comes from a Wegman-Carter tag whose one-time mask is taken fresh from the same key stream, the hash key is reused across records but the mask is not, which costs 16 key bytes per record on top of the payload. Every record is sealed the same way, shown in Listing 4, consuming `len(payload) + 16` key bytes in a fixed order, so both ends consume the same key bytes in the same order and stay synchronised without exchanging any extra information.

```
function seal(rtype, plaintext):
  ciphertext <- plaintext XOR key_stream.take(len(plaintext))
  mask      <- key_stream.take(16)
  header    <- pack(rtype, len(ciphertext)) // record type +
length
  tag       <- wegman_carter(header + ciphertext, mask)
  return header + ciphertext + tag
```

Listing 4: Sealing one record: one-time-pad encryption followed by a Wegman-Carter tag, consuming payload-length plus 16 key bytes.

Environment and setup

The whole project is a single Python package, so setting it up takes one command after creating a virtual environment, and a Makefile wraps the common tasks so none of them have to be typed by hand. `make setup` creates the environment and installs the package together with its optional groups (the test, dashboard and benchmark dependencies). After that, `make demo` runs the command-line chat demo, `make dash` launches the live dashboard in the browser, `make test` runs the test suite, and `make bench` runs the quick benchmark profile. The only requirement beyond Python is a C compiler, used once to build the Cascade extension during installation. The project requires Python 3.12 or newer and Table 2 lists the main libraries and the minimum versions the build pins.

Table 2. Main dependencies and the minimum versions pinned by the build.

Library	Min. version	Used for
Qiskit / qiskit-aer	1.2 / 0.15	BB84 circuit simulation
NumPy	1.26	bit packing in reconciliation
FastAPI	0.115	ETSI 014 REST interface
Pydantic	2.8	request/response validation
httpx	0.27	SAE-side KME client
cryptography	43	HKDF, SHA-2, X.509
Streamlit / Plotly	1.31 / 5.20	live dashboard
pyperf	2.6	benchmark timing
pytest	8.0	test suite

5. Evaluation

Evaluation strategy

The evaluation first checks whether the system works at all: whether the two ends really agree on the same key, whether a key is never reused, whether the KME interface stays faithful to ETSI 014, and whether an end-to-end session can be established and carry data. The other thing we look at, and the more interesting one, is what all this costs. Because the thesis trades a computational guarantee for an information-theoretic one, the point is not to beat classical TLS on speed but to measure the price of that trade. Therefore, the tests measure handshake latency and time-to-last-byte, application-data throughput, the rate at which the key pool drains under load, the amount of key consumed per byte of data, and how the final key rate behaves as the channel error rate rises. The baselines which are compared with QTLS are experiments chosen on aspects where it would make sense to measure the trade-off. The handshake latency is compared directly against a real TLS 1.3 handshake, and the data-path throughput against TLS 1.3 with AES-GCM, since both protocols do the same job there. The key exchange is compared three ways: the QKD key fetch against the X25519 elliptic-curve exchange and against the post-quantum ML-KEM-768, on latency and on bytes sent over the wire. The remaining three experiments, pool drain, key consumption and the QBER sweep, have no baseline because they measure properties that exist only because the keys come from a finite QKD supply.

Benchmarks and metrics

The benchmark suite consists of six modular experiments, each measuring one cost and runnable on its own.

The handshake-latency experiment measures the time to complete a QTLS handshake against a real TLS 1.3 handshake, reported as median, 95th and 99th percentile over many runs.

The throughput experiment measures the sustained application-data rate of the one-time-pad data path against TLS 1.3 with AES-GCM, across several payload sizes.

The pool-drain experiment measures how many handshakes, records and bytes a KME key pool sustains under continuous load before it is exhausted, given a fixed key-generation rate.

The key-consumption experiment measures the ratio of key bytes consumed to useful payload bytes as a function of record size, which exposes the fixed 16-byte authentication overhead per record.

The QBER sweep measures the final secret-key rate as a function of the quantum bit error rate, showing how reconciliation leakage and privacy amplification shrink the usable key as the channel degrades.

Finally, the key-exchange experiment measures the latency and the on-the-wire size of obtaining a shared secret, comparing the QKD key fetch against X25519 and ML-KEM-768.

Experimental setup

All measurements come from a single run of the `full` benchmark profile, which fixes the iteration counts for every experiment: the handshake is measured over 5000 timed runs after a warm-up, the key exchange over 5000, and the throughput, pool-drain and QBER experiments over fixed time budgets and payload lists.

Across all experiments the keys come from the abstract high-rate generator rather than from the quantum BB84 simulation, which is too slow to produce key at these volumes; the implications of this choice are discussed in the discussion chapter. The generator still runs the same reconciliation and privacy-amplification pipeline as BB84, so the keys it produces are cryptographically identical and only the raw-entropy source differs.

The whole stack runs in one process on a laptop with an Intel Core i7-10810U CPU (6 cores, 12 threads) and 32 GB of RAM, running Ubuntu 24.04 and Python 3.12, so the figures presented below reflect the real protocol and record-layer code path the earlier chapters describe, not a model of it. The TLS 1.3 baseline is a real handshake and record layer over the same loopback transport, so the two protocols are measured under identical conditions and only the cryptography differs. The library versions are those pinned in Table 2.

Results

Handshake latency

Figure 4 compares the two handshakes. The QTLS handshake takes a median of about 44.6 ms against 2.9 ms for TLS 1.3, roughly an order of magnitude slower. That gap comes from round-trips: a QTLS handshake makes two extra requests to the KME (the master pull and the slave fetch) before any traffic flows, whereas TLS 1.3 derives its secret inline from the Diffie-Hellman exchange. The tail (99th percentile) is influenced by those requests rather than by the one-time-pad work, which is negligible.

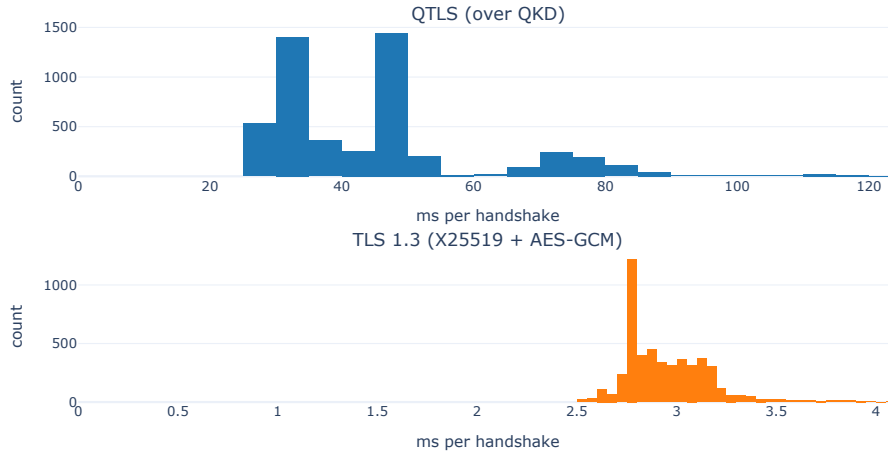


Figure 4. Handshake latency distribution, QTLS versus TLS 1.3. The two histograms use independent x-axes, and the 99.5th percentile is clipped.

Throughput

Figure 5 shows sustained throughput across payload sizes. Here QTLS is much closer to TLS 1.3 than the handshake suggests, because a one-time pad is a single XOR per byte, cheaper than the block-cipher work in AES-GCM. And the cost of QTLS on the data path is the key budget it spends, not the time it takes.

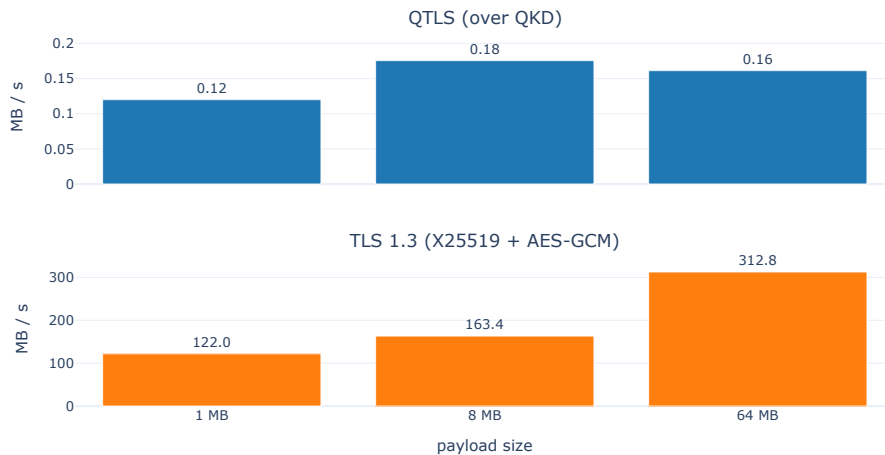


Figure 5. Application-data throughput, QTLS versus TLS 1.3.

Pool drain

Figure 6 shows the key pool under a paced, chat-style load. Starting from a small prefill, the pool climbs steadily for the whole run because the QKD producer outpaces this light consumption by a wide margin: at a realistic chat rate the source keeps up easily, so the pool only grows. The cumulative payload on the right axis rises in steps as messages are sent,

but the key this consumes is too small against the production rate to bend the pool curve downward. The two KME curves coincide because the feeder writes the same blocks into both stores. The takeaway is that a normal chat never exhausts the pool. Only a saturating sender, one that ships data faster than the source refills the pool, drains it to zero; at that point the sender is throttled to the key-production rate, since it cannot send a byte it has no key for. The rate at which keys are produced is the parameter that decides which regime a deployment is in.

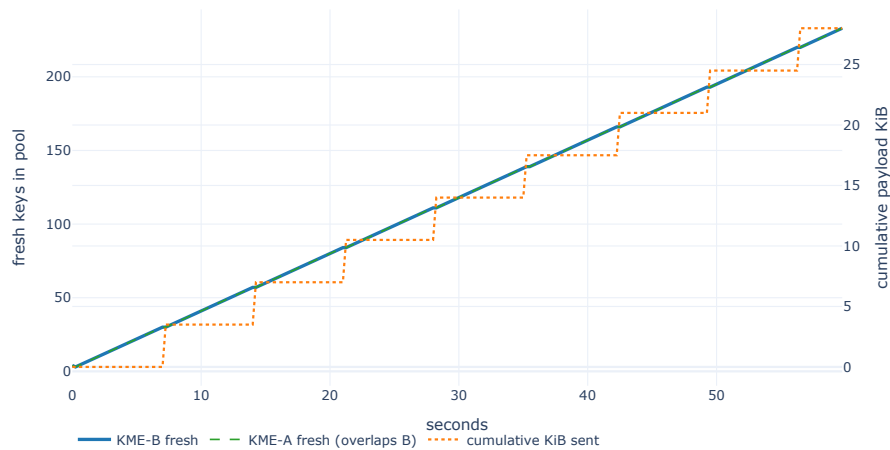


Figure 6. Key pool over time. The two y-axes are independent: the left counts keys in the pool (each key is an 8 KiB block), the right is the cumulative payload sent in KiB.

Key consumption

Figure 7 plots the ratio of key bytes consumed to useful payload bytes against record size. The ratio is dominated by the fixed 16-byte Wegman-Carter mask per record: for small records the overhead is large, while for large records it amortises and the ratio approaches one key byte per payload byte.

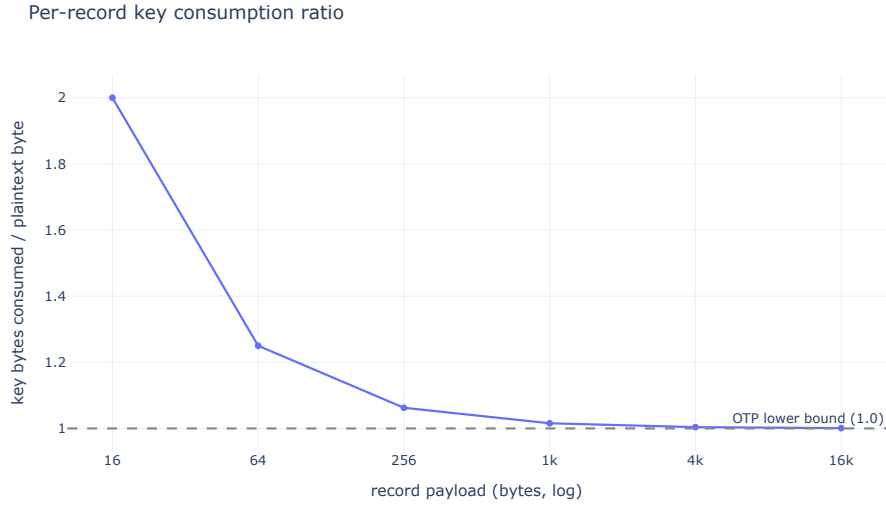


Figure 7. Key consumed per useful byte, as a function of record size.

Key rate versus QBER

Figure 8 shows the final secret-key rate as the quantum bit error rate rises. The rate falls as QBER grows, because more errors mean more parity bits revealed during reconciliation and a larger reduction during privacy amplification. Above the Shor-Preskill abort threshold, introduced in the background chapter, no key is produced at all.

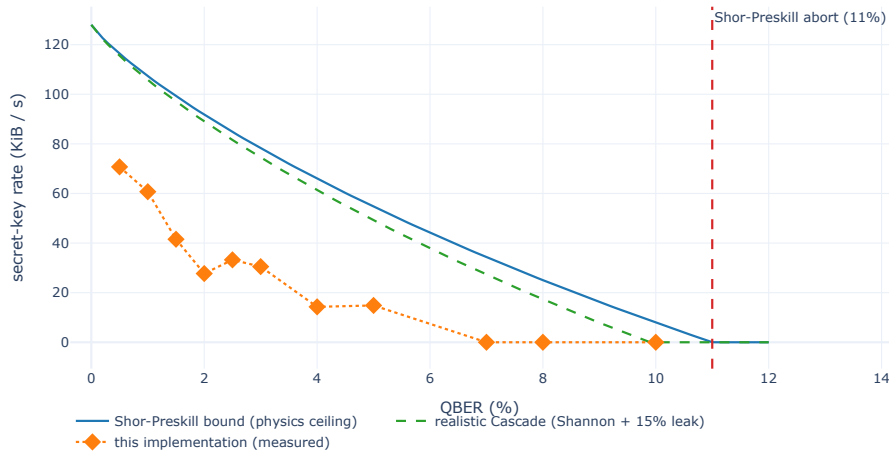


Figure 8. Final secret-key rate as a function of the channel QBER (asymptotic rate at a 1.0 Mbps raw key rate).

Key exchange comparison

Figure 9 and Figure 10 compare obtaining the shared secret three ways. In latency the QKD fetch is the slowest (a median of about 2336 us against 255 us for X25519 and 343 us for ML-KEM-768), because it is a network request to the KME rather than a local computation. On the link, however, the picture inverts: the QKD path sends only a 16-byte key identifier

and never the key itself, against 32 bytes each way for X25519 and over a kilobyte each way for ML-KEM-768.

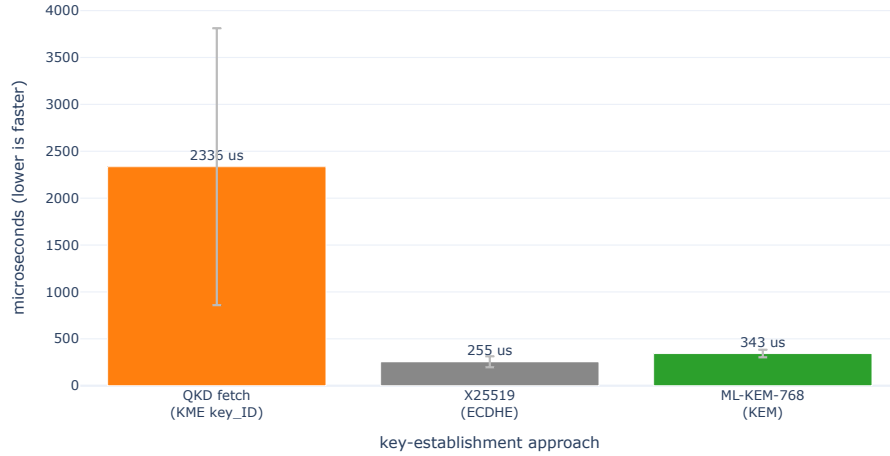


Figure 9. Key-exchange latency: QKD fetch, X25519, ML-KEM-768 (median with 95th-percentile error bars).

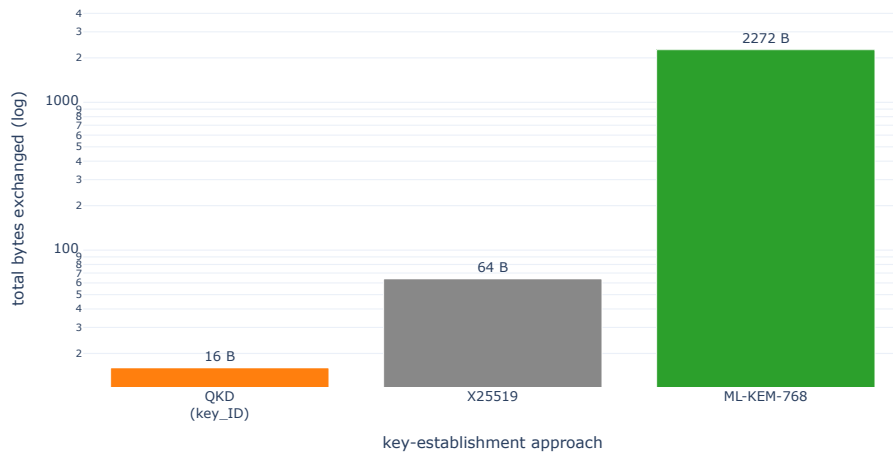


Figure 10. On the link bytes per key exchange (log scale).

Data analysis

Read together, the experiments confirm that QTLS does not win on speed: its handshake is about an order of magnitude slower than TLS 1.3, and its key exchange is several times slower than either X25519 or ML-KEM-768. Both slowdowns come from the round-trips to the KME, not from the information-theoretic primitives, which are cheap: the one-time pad keeps throughput close to TLS, and the per-record cost is a flat 16 bytes.

The real constraint on QTLS is not time but key material: the one-time pad spends a byte of key for every byte of data, so a session's reach is bounded by how much key it holds, and the supply itself shrinks as the channel error rate rises. The implementation makes this limit visible by letting a session end once its pool is exhausted instead of silently fetching more, which keeps the cost of QKD visible throughout the measurements. QTLS pays in a finite key budget, and the experiments show that this is a price worth paying where the guarantee has to hold regardless of the attacker's power.

Against the alternatives the trade is sharpest on the communication link. A post-quantum key exchange such as ML-KEM-768 keeps TLS fast but inflates the handshake to more than a kilobyte in each direction, which is exactly the cost the data-heavy TLS study [30] measures while the QKD path sends 16 bytes. The comparison is not entirely like-for-like, since ML-KEM is measured as a primitive and its security is still computational, but it frames the difference: post-quantum cryptography buys quantum resistance cheaply and keeps a computational assumption, whereas QKD removes the assumption and pays for it in latency and key budget.

Evaluation conclusion

The evaluation confirms that the system does what it claims: the two ends agree on the same key, the key is used once, the ETSI interface is honoured, and a session runs end to end with information-theoretic protection on the data path. It also quantifies the cost of that guarantee. QTLS is slower to start and bounded by its key supply, but its data path is competitive and the bytes used for key exchange are lower. This answers the question the thesis set out with: unconditional security on a standard secure-channel protocol is achievable, and its price is measurable, concentrated in handshake latency and key consumption rather than in the protection of the data itself.

6. Discussion

Summary of key findings

Our evaluation confirmed that the QTLS handshake is slower than classical TLS 1.3. The reason is the round-trips to the Key Management Entity, not the cryptography itself. Apart from that, the system works: it sets up a session whose data path is protected information-theoretically and the experiments also measured what this protection costs. Each byte of payload spends one byte of QKD key, and every record carries a fixed 16-byte authentication tag on top. Finally, they showed that the channel error rate controls how much key gets produced, with a threshold above which no key is produced at all.

Interpretation of results

The handshake is slow for an infrastructural reason. The KME is reached over HTTP, and fetching the key identifiers takes a few round-trips, a cost a session pays only once, at startup. That is acceptable here, because QTLS is meant for short, important exchanges, not for bulk traffic, so paying tens of milliseconds once does not hurt.

On the data path the cost is built into the design, and it comes from the one-time pad. Encrypting one byte of payload consumes one byte of key, and each record adds a fixed 16-

byte mask on top, so the amount a session can send is bounded directly by the amount of key it holds. This makes communication depend on the key-generation rate rather than on processing power. For light traffic the bound is not a real constraint: as long as the sender does not consume key faster than the link produces it, the supply is never exhausted. In a real deployment this is the normal case, since a QKD link continuously fills a large pool of generated keys that the SAEs draw from on demand, and a saturating sender that outpaces that production is the only situation in which the budget actually runs out.

One caveat on that experiment: the key producer used there is a fast random key generator, not a real QKD link. So it shows the drain versus refill behaviour, and it is modelled to produce keys at a rate a standard QKD would. Whether a deployment stays in the comfortable regime depends entirely on the real key-generation rate.

The last result is the link between error rate and key rate. As errors rise, the key rate falls, and what is above the Shor-Preskill threshold drops to zero. It is exactly the mechanism that makes the security unconditional: if the error rate is too high, an eavesdropper might be listening, so the protocol refuses to produce a key instead of producing one that could be compromised.

One point applies to all of these numbers. The benchmarks draw their keys from the abstract high-rate generator, not from the quantum BB84 simulation, so they measure the protocol and the post-processing, not the cost of producing key on real quantum hardware. The keys are not weaker for it, since the abstract generator runs the same reconciliation and privacy-amplification pipeline as BB84, but the figures should be read as the cost of everything above the quantum layer rather than of the full system.

Comparison with existing work

These findings line up with Rubio Garcia et al. [16], who chose not to use a one-time pad on the TLS data path precisely to save QKD key material. Our results extend their position and show that, in pure computational terms, an OTP record layer is cheap and practical: a single XOR per byte keeps throughput close to TLS. By accepting the strict one to one key consumption on purpose, this thesis demonstrates that end-to-end information-theoretic security inside a standard protocol is feasible in a prototype, and that the hard part is the key-generation rate of the hardware, not the software.

The contrast with post-quantum cryptography sharpens this. As the key-exchange comparison showed, a scheme like ML-KEM-768 keeps TLS fast and easy to deploy but leaves a computational assumption in place. QKD takes the opposite trade: it removes the assumption entirely and pays for it in latency, key budget, and infrastructure. It is important to understand the scope of the problem. The quantum-distributed key is secure in an information-theoretic sense and should be used on important data, while ML-KEM is a good alternative against a quantum computer but that guarantee can still be broken.

Implications for the domain

For the wider area of secure communications, QTLS is not a general-purpose protocol, and it does not try to be. It fits a niche: short, highly critical messages that would be exposed by a harvest-now-decrypt-later attack, where someone records your traffic today and just

waits for a quantum computer or algorithm to break its security. Things like cryptographic bootstrapping keys or other secrets that have to stay confidential for decades. For that kind of payload, paying in latency and key budget is an acceptable trade. The broader implication is that it makes a case for building more information-theoretically secure systems where the stakes justify them, and it highlights a real problem: anything resting on a computational assumption today is only as safe as that assumption stays unbroken, and a future attack on it would expose every message ever recorded under it.

The protocol itself delivers the security it promises, but what holds it back is not the cryptography but how hard it is to adopt, since it asks for a whole QKD infrastructure that few organisations have today. That is an argument for encouraging this direction rather than dismissing it: work that makes the case for unconditional security and mentions its importance is what will decide whether protocols like QTLS ever move beyond a niche.

Limitations

Several limitations of the current prototype should be acknowledged. The architecture models a point-to-point link between exactly two endpoints, which limits its range because the quantum signal degrades over long optical fibre. It also relies on the trusted-node assumption from the ETSI 014 standard, and this is a genuine gap in the security model, not a detail: the KME sees the key in the clear. The data path is information-theoretically secure, but each KME is a point that has to be trusted, so the unconditional guarantee holds between endpoints only as far as the KMEs are honest.

On the implementation side, privacy amplification uses SHAKE-128 as a practical extractor instead of a mathematically pure universal hash family. The modular design makes this a drop-in replacement, but as it stands it softens the strictness of the information-theoretic claim. The KME synchronisation through a shared lock and the header-simulated mTLS are simplifications that fit a local proof-of-concept, not a distributed production deployment.

Beyond the prototype, the protocol is fundamentally constrained by its hardware. Its security depends on a real QKD link, a KME, and the SAEs, so the guarantee is only as available as that infrastructure is. A deployment needs all of it in place and producing key at a usable rate, which is expensive and not yet widely accessible, and this is as much a limit on where QTLS can be used as any property of the protocol itself.

Future work

Future work should close the theoretical gap first, by replacing the SHAKE-128 extractor with a Toeplitz-matrix construction so that privacy amplification is unconditionally secure. SHAKE-128 was used because it is a strong cryptographic extractor in practice and was enough to sustain the privacy of the key; the Toeplitz construction would tighten the theoretical guarantee rather than fix a functional gap. On the infrastructure side, the KME synchronisation should move to a distributed two-phase commit protocol with real network-level mTLS instead of the header simulation.

Two further extensions would lift the current limits on distance and data volume. The architecture could be integrated with multi-hop QKD networks for longer-range routing, which would relax the two-endpoint constraint. A mid-session key-refetching mechanism would

let an active connection draw fresh keys before its pool runs out, so a session is no longer bounded by the key it held at startup.

A few smaller extensions follow directly from the gaps above. The ETSI 014 interface lets a master SAE share one key with several slave SAEs at once, which the current prototype does not support, so adding this multi-SAE delivery would let the same key reach a group rather than a single peer. The benchmarks could also be re-run against a real QKD source instead of the abstract generator, for example on the quantum infrastructure available at the faculty, to measure the cost of the quantum layer that the current figures leave out and complete the picture the evaluation gives.

7. Conclusions

This thesis delivered a secure-channel stack whose data path does not rest on any computational assumption. Between the two endpoints, confidentiality comes from a one-time pad, while integrity and authentication come from a Wegman-Carter tag and from proving possession of the same quantum-distributed key, all keyed from QKD material rather than from any problem being hard to solve. Where TLS 1.3 protects almost all web traffic today by relying on problems like the elliptic-curve discrete logarithm staying hard to solve, the system built here removes that dependency entirely. The two endpoints draw identical key material from a quantum key distribution link instead of negotiating it over the network, so the secrecy of the channel comes from the laws of physics rather than from the cost of a computation.

What was built is a complete system from point A to B. Key material is produced over a QKD link and reconciled into final secret bytes, a key management entity stores and serves it through the ETSI 014 interface, and two secure application entities run a TLS-style handshake and then protect every record with a one-time pad and a Wegman-Carter authentication tag.

The point of the comparison with classical and post-quantum alternatives is not that QTLS is faster, because it is not, and it never tried to be. Its advantage is the security model: even if it is slower, the fact that the data stays secure for as long as it must is priceless. A post-quantum scheme such as ML-KEM-768 keeps TLS fast and easy to deploy, but it still rests on a computational assumption. QTLS takes the opposite trade: it drops the assumption entirely and pays for that in latency and in key budget. Where the data must stay secret regardless of how much computing power an attacker ever gains, that is the trade worth making. Even though it serves a niche, it proves its worth by protecting critical information under a strong theoretical guarantee.

The evaluation worked both as validation and as a benchmark. First, the system does what it claims: the two ends agree on the same key, each key is used once, the ETSI interface is honoured, and a session runs end to end with information-theoretic protection on the data path. Second, the cost of that guarantee is measurable and sits in a predictable place. It is concentrated in the handshake, which is about an order of magnitude slower than TLS because of the round-trips to the KME, and in the key budget, since the one-time pad spends

a byte of key for every byte of data. The data path itself stays competitive, so the price is paid at setup and in key consumption, not in the protection of the data.

In developing this work, the strongest constraint and the one where other expertise is needed is hardware: a full deployment needs real QKD endpoints, optical fibre, and the supporting infrastructure, none of which was available for this work. The measurements therefore use an abstract high-rate key generator and a BB84 simulation in place of a physical link, which means the cost of the quantum layer itself is outside what we could measure, and the behaviour of the channel over real long-distance fibre could not be tested. These are limits of access and resources rather than flaws in the design, and they bound what the evaluation can claim more than they bound the protocol.

Several extensions are left for the future: replacing the SHAKE-128 extractor with a Toeplitz construction to tighten the theoretical guarantee, moving KME synchronisation to a distributed protocol with real mutual TLS, supporting multi-hop QKD networks to reach beyond two endpoints, and adding the option of mid-session key refetching so a connection is not bounded by the key it starts with. Re-running the benchmarks against a real QKD source, for instance on the quantum infrastructure available at the faculty, would also close the gap left by the abstract generator. These directions are discussed in more detail in the discussion chapter.

The question this thesis set out to answer was whether unconditional security could be placed on a standard secure-channel protocol, and the answer is that it can. It comes at a cost, since the handshake is slower than TLS and every byte of data spends a byte of key, but that cost buys something no computational scheme can offer: protection on the data path that holds regardless of how much computing power an attacker ever gains. QTLS is not a replacement for TLS on everyday traffic, yet for short, high-value data that has to stay secret for decades, that guarantee is what makes it worth the price.

References

- [1] E. Rescorla, “The Transport Layer Security (TLS) Protocol Version 1.3,” RFC 8446, 2018. doi: 10.17487/RFC8446.
- [2] Google, “HTTPS Encryption on the Web (Google Transparency Report).” 2026.
- [3] D. J. Bernstein, J. Buchmann, and E. Dahmen, Eds., *Post-Quantum Cryptography*. Springer, 2009. doi: 10.1007/978-3-540-88702-7.
- [4] National Institute of Standards and Technology, “NIST SP 800-186: Recommendations for Discrete Logarithm-Based Cryptography: Elliptic Curve Domain Parameters,” Special Publication 800–186, 2023. doi: 10.6028/NIST.SP.800-186.
- [5] N. Koblitz, “Elliptic Curve Cryptosystems,” *Mathematics of Computation*, vol. 48, no. 177, pp. 203–209, 1987, doi: 10.1090/S0025-5718-1987-0866109-5.

- [6] P. W. Shor, “Algorithms for Quantum Computation: Discrete Logarithms and Factoring,” in *Proceedings 35th Annual Symposium on Foundations of Computer Science*, IEEE, 1994, pp. 124–134. doi: 10.1109/SFCS.1994.365700.
- [7] M. Mosca, “Cybersecurity in an Era with Quantum Computers: Will We Be Ready?,” *IEEE Security & Privacy*, vol. 16, no. 5, pp. 38–41, 2018, doi: 10.1109/MSP.2018.3761723.
- [8] National Institute of Standards and Technology, “FIPS PUB 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard.” 2024. doi: 10.6028/NIST.FIPS.203.
- [9] ETSI Industry Specification Group on Quantum Key Distribution, “Quantum Key Distribution (QKD); Protocol and Data Format of REST-Based Key Delivery API,” GS QKD14V1.1.1, 2019. [Online]. Available: https://www.etsi.org/deliver/etsi_gs/QKD/001_099/014/01.01.01_60/gs_QKD014v010101p.pdf
- [10] C. E. Shannon, “Communication Theory of Secrecy Systems,” *Bell System Technical Journal*, vol. 28, no. 4, pp. 656–715, 1949, doi: 10.1002/j.1538-7305.1949.tb00928.x.
- [11] M. N. Wegman and J. L. Carter, “New Hash Functions and Their Use in Authentication and Set Equality,” *Journal of Computer and System Sciences*, vol. 22, no. 3, pp. 265–279, 1981, doi: 10.1016/0022-0000(81)90033-7.
- [12] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, 10th Anniversary. Cambridge University Press, 2010. [Online]. Available: <https://www.cambridge.org/highereducation/books/quantum-computation-and-quantum-information/01E10196D0A682A6AEFFEA52D53BE9AE>
- [13] Electronic Frontier Foundation, *Cracking DES: Secrets of Encryption Research, Wiretap Politics, and Chip Design*. Sebastopol, CA: O'Reilly, Associates, 1998.
- [14] X. Wang and H. Yu, “How to Break MD5 and Other Hash Functions,” in *Advances in Cryptology – EUROCRYPT 2005*, in Lecture Notes in Computer Science, vol. 3494. Springer, 2005, pp. 19–35. doi: 10.1007/11426639_2.
- [15] M. Stevens, E. Bursztein, P. Karpman, A. Albertini, and Y. Markov, “The First Collision for Full SHA-1,” in *Advances in Cryptology – CRYPTO 2017*, in Lecture Notes in Computer Science, vol. 10401. Springer, 2017, pp. 570–596. doi: 10.1007/978-3-319-63688-7_19.
- [16] C. Rubio García *et al.*, “Quantum-Resistant Transport Layer Security,” *Computer Communications*, vol. 213, pp. 345–358, 2024, doi: 10.1016/j.comcom.2023.11.010.
- [17] D. Stebila, S. Fluhrer, and S. Gueron, “Hybrid Key Exchange in TLS 1.3.” [Online]. Available: <https://datatracker.ietf.org/doc/draft-ietf-tls-hybrid-design/>
- [18] H. Krawczyk and P. Eronen, “HMAC-Based Extract-and-Expand Key Derivation Function (HKDF),” RFC 5869, 2010. doi: 10.17487/RFC5869.

- [19] G. S. Vernam, “Cipher Printing Telegraph Systems for Secret Wire and Radio Telegraphic Communications,” *Journal of the AIEE*, vol. 45, no. 2, pp. 109–115, 1926, doi: 10.1109/JAIEE.1926.6534724.
- [20] J. L. Carter and M. N. Wegman, “Universal Classes of Hash Functions,” *Journal of Computer and System Sciences*, vol. 18, no. 2, pp. 143–154, 1979, doi: 10.1016/0022-0000(79)90044-8.
- [21] D. R. Stinson, “Universal Hashing and Authentication Codes,” *Designs, Codes and Cryptography*, vol. 4, no. 3, pp. 369–380, 1994, doi: 10.1007/BF01388651.
- [22] P. W. Shor and J. Preskill, “Simple Proof of Security of the BB84 Quantum Key Distribution Protocol,” *Physical Review Letters*, vol. 85, no. 2, pp. 441–444, 2000, doi: 10.1103/PhysRevLett.85.441.
- [23] L. K. Grover, “A Fast Quantum Mechanical Algorithm for Database Search,” in *Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC)*, 1996, pp. 212–219. doi: 10.1145/237814.237866.
- [24] W. Castryck and T. Decru, “An Efficient Key Recovery Attack on SIDH,” in *Advances in Cryptology — EUROCRYPT 2023*, in Lecture Notes in Computer Science, vol. 14008. Springer, 2023, pp. 423–447. doi: 10.1007/978-3-031-30589-4_15.
- [25] C. H. Bennett and G. Brassard, “Quantum Cryptography: Public Key Distribution and Coin Tossing,” in *Proceedings of IEEE International Conference on Computers, Systems and Signal Processing*, Bangalore, India, 1984, pp. 175–179.
- [26] G. Brassard and L. Salvail, “Secret-Key Reconciliation by Public Discussion,” in *Advances in Cryptology — EUROCRYPT '93*, in Lecture Notes in Computer Science, vol. 765. Springer, 1994, pp. 410–423. doi: 10.1007/3-540-48285-7_35.
- [27] J. Martínez-Mateo, C. Pacher, M. Peev, A. Ciurana, and V. Martín, “Demystifying the Information Reconciliation Protocol Cascade,” *Quantum Information & Computation*, vol. 15, no. 5–6, pp. 453–477, 2015, doi: 10.26421/QIC15.5-6-3.
- [28] C. H. Bennett, G. Brassard, C. Crépeau, and U. M. Maurer, “Generalized Privacy Amplification,” *IEEE Transactions on Information Theory*, vol. 41, no. 6, pp. 1915–1923, 1995, doi: 10.1109/18.476316.
- [29] R. Renner, “Security of Quantum Key Distribution,” Doctoral dissertation, 2005. doi: 10.3929/ethz-a-005115027.
- [30] P. Kampanakis and W. Childs-Klein, “The impact of data-heavy, post-quantum TLS 1.3 on the Time-To-Last-Byte of real-world connections,” in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2024. [Online]. Available: <https://eprint.iacr.org/2024/176>
- [31] Python Software Foundation, “asyncio — Asynchronous I/O.” [Online]. Available: <https://docs.python.org/3/library/asyncio.html>

- [32] Qiskit contributors, “Qiskit: An Open-Source Framework for Quantum Computing.” 2024. doi: 10.5281/zenodo.2573505.
- [33] S. Ramírez, “FastAPI.” [Online]. Available: <https://fastapi.tiangolo.com/>
- [34] Pydantic contributors, “Pydantic: Data validation using Python type hints.” [Online]. Available: <https://docs.pydantic.dev/>
- [35] Encode OSS, “HTTPX: A next-generation HTTP client for Python.” [Online]. Available: <https://www.python-httpx.org/>
- [36] Python Cryptographic Authority, “cryptography.” [Online]. Available: <https://cryptography.io/>
- [37] Snowflake Inc., “Streamlit: A faster way to build and share data apps.” [Online]. Available: <https://streamlit.io/>
- [38] Plotly Technologies Inc., “Plotly Python Open Source Graphing Library.” [Online]. Available: <https://plotly.com/python/>
- [39] V. Stinner, “pyperf: Toolkit to run Python benchmarks.” [Online]. Available: <https://pyperf.readthedocs.io/>

Appendices

[Fill in the content of this chapter.]